

Práctica 1 - Diseño de plataforma robótica y control de los motores.

Alejandro Otero Vázquez

Fecha de realización: 13 de enero de 2025

1. Montaje del robot

1.1. Materiales

Para el montaje de este robot han sido necesarios dos servos de rotación continua que impulsarán las ruedas motrices. El eje delantero está compuesto por una rueda de roll-on capaz de girar sobre su eje vertical para que el robot pueda girar en todas direcciones. Como microcontrolador se ha empleado una RaspberryPi3. Esta placa controlará la velocidad y dirección del robot mediante dos pines PWM conectados a sendos servos. Tanto los motores como la placa están alimentadas con una powerbank capaz de suministrar hasta 22,5W.

Tanto los motores, como la RaspberryPi pertenecen a un kit que no se incluirá en el presupuesto de montaje. A continuación se adjunta una tabla con precios y referencias donde adquirir los componentes que se han empleado en la construcción del robot.

Presupuesto Robot Diferencial		
Tabla de madera (21cmx18cm)	9.99 €	https://www.bauhaus.es/tableros-de-madera-laminada/exclusivholz-tablero-de-madera-laminada/p/20509596
Tabla de madera (10cmx18cm)		
Powerbank 13800 mAh	15.99 €	https://www.amazon.es/dp/B0CPI3QK21?psc=1&ref=ppx_yo2ov_dt_b_product_details
Rueda roll-on	5.49 €	https://www.bauhaus.es/ruedas-giratorias/stabilit-rueda-giratoria-para-equipos/p/10137320

Figura 1: Precios y referencias de los materiales utilizados para el montaje del robot.

1.2. Ensamblaje de los motores

Un correcto alineamiento del eje de los motores es tremendamente importante para que el robot sea capaz de circular sin desviarse. Si los motores no conservan un eje común de rotación durante toda la trayectoria, las ruedas motrices sufrirán deslizamientos indeseados. Este mal funcionamiento puede hacer imposible controlar la trayectoria del robot, dificultando tremendamente que el dispositivo sea capaz de recorrer una distancia en línea recta.

Hay que ser conscientes siempre de con qué tipo de elementos se está trabajando. Los servomotores están controlados por señales PWM, que no siempre tienen una correlación exacta entre señal y comportamiento. Es posible que un motor necesite un duty-cycle ligeramente distinto que otro para girar a la misma velocidad. Es muy importante calibrar el potenciómetro de los servos para que la relación de la velocidad angular con la señal PWM sea la misma. Otro factor que favorece la desviación del robot es el deslizamiento que sufren las ruedas con el pavimento. Esto es algo inevitable, al igual que los errores de cuantización derivados del PWM, y hará que el robot típicamente sea incapaz de mantenerse en una trayectoria recta indefinidamente. En definitiva, existen gran cantidad de factores ambientales que afectan al correcto funcionamiento del móvil. Es por esto que alinear las ruedas motrices y calibrar los servomotores es la base para construir una buena

estructura que permita controlar en el futuro el dispositivo.

Con todo esto, entre las dos placas de madera queda disponible un espacio en el que se situará la powerbank

Típicamente un servomotor tiene un aspecto similar al que se muestra en la figura 2, con 4 orificios para atornillarlo a una superficie. Estos servos presentan unas dimensiones de 20mmx40mmx40mm. Una tabla de 20 mm de ancho no es lo suficientemente consistente como para soportar 4 tornillos de esa métrica. Además la madera con la que está construido el robot es laminada por lo que perforarla en la dirección en que están apiladas las láminas destruye su consistencia. Por esta razón, se ha optado por ensamblar los motores entre dos tablas de mayor tamaño. Estas tablas, acopladas en forma de sándwich con los motores, ejercen una presión suficiente como para fijar los motores en su posición indefinidamente. El par que puedan generar estos servos no es suficiente como para desalinearlos de su posición venciendo la presión de las tablas.

Siguiendo la orientación de la figura 2, las tablas se situarían a la izquierda y a la derecha del servomotor, fijando su posición. Los ejes de ambos motores (substituyendo la cruz por una rueda) quedarían fuera de la estructura permitiendo así su rotación.



Figura 2: Aspecto típico de un servomotor.

y la mayoría de cables que puedan ir escondidos. Esto tiene dos ventajas: la primera es que baja el centro de gravedad del sistema, lo que lo hace más estable. La segunda ventaja es la limpieza y claridad que le da a los circuitos que irán situados sobre la tabla superior. A medida que el proyecto avance, cada vez los circuitos irán siendo más complejos y por lo tanto serán necesarios muchos cables. El tener un espacio interior que oculte la mayoría de ellos reduce las probabilidades de que alguno se suelte y da una imagen al robot mucho más limpia y profesional.

Por otra parte, y para finalizar con este apartado. El eje delantero está formado simplemente por una rueda giratoria de 50mm de altura, que nivela perfectamente la estructura y sirve como punto de apoyo del sistema. Su alineación no es tan relevante en el movimiento como la del eje trasero pero sí que es muy recomendable situarla lo más centrada posible para darle estabilidad a la plataforma.

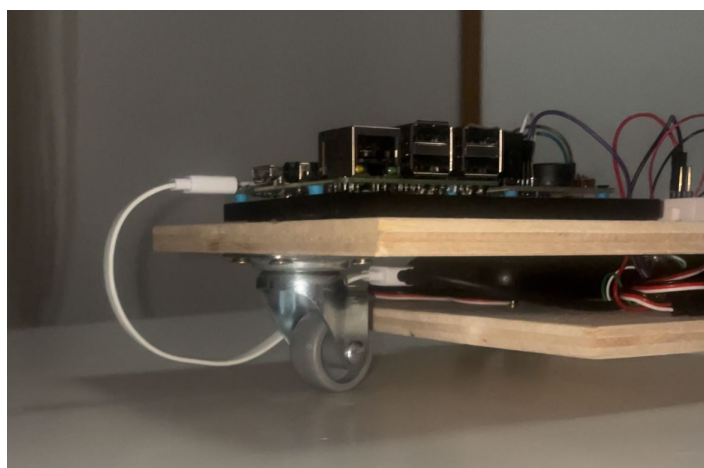


Figura 3: Vista delantera del robot.

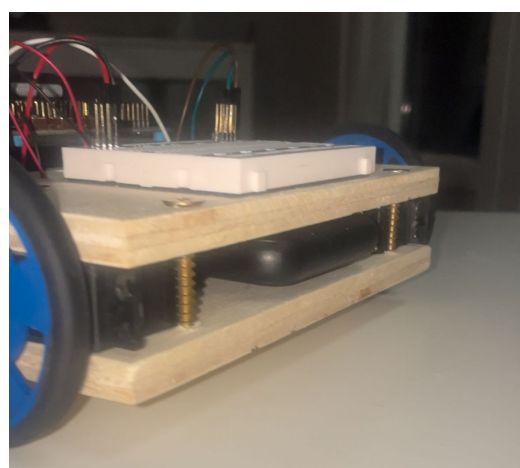


Figura 4: Vista del eje trasero del robot.

1.3. Electrónica y control del robot

En la parte superior del robot se sitúan el microcontrolador que controlará todo el software y computación del dispositivo y un entrenador. Este último es útil dada la gran cantidad de conexiones que manejará el robot en un futuro cuando se incluyan los sensores. La powerbank tiene dos salidas usb que alimentarán los circuitos. Una de ella se conectará por micro-usb a la alimentación de la raspberry, mientras que del otro conector usb se sacarán dos cables que alimentarán dos líneas del entrenador a las que irán conectados los motores. Para ello ha de pelarse el cable usb, que típicamente está formado por 4 cables, dos tierras y dos positivos. Un par de tierra positivo es el de alimentación y el otro el de bus de datos. Para esta aplicación nos quedaremos solo con los cables de alimentación.

Como se observa en la figura 5, de han hecho una serie de orificios en una de las tablas para pasar todos los cables relativos a los motores por ahí y que el robot tenga un aspecto mucho más limpio.

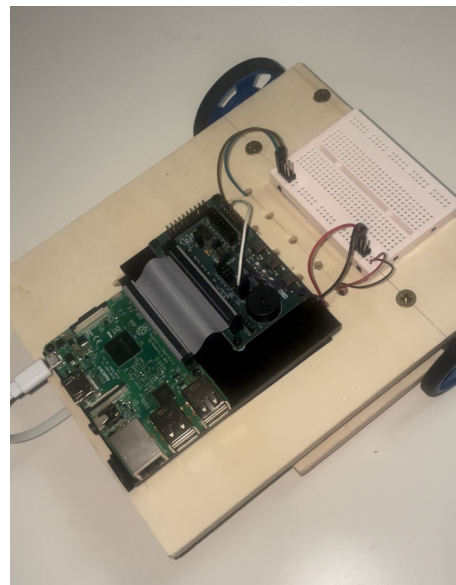


Figura 5: Vista delantera del robot.

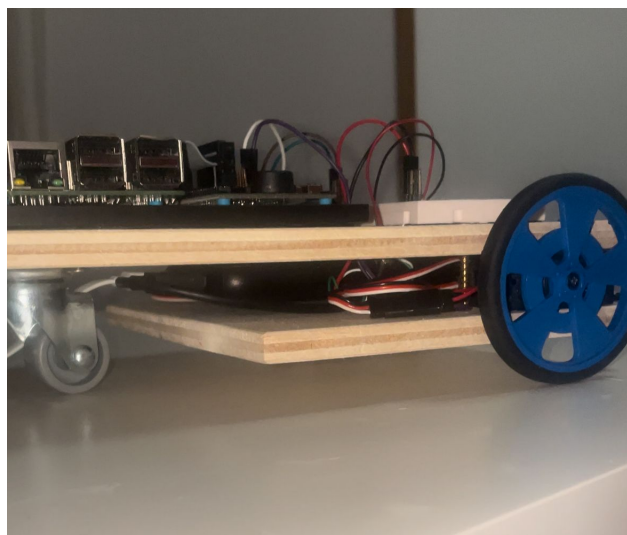


Figura 6: Vista lateral del robot.

Por último, siempre es útil hacer un repaso ante los problemas o dificultades más reseñables que uno se puede encontrar en caso de hacer un robot diferencial que monte este hardware. El mayor reto que el diseñador afronta cuando construye un robot de este tipo es conseguir una trayectoria recta a largas distancias. Con los materiales que se ha montado este robot, dado el presupuesto del que se disponía es imposible recorrer más de 3 metros en línea recta sin que el desvío comience a ser notorio. Al fin y al cabo la velocidad de los motores es muy sensible a la posición del potenciómetro que integran. Este potenciómetro debe ajustarse con un tornillo y se calibra para que cuando el duty-cycle del PWM sea de 1.5/20 el motor no gire.

Esta calibración no asegura ni mucho menos que para valores del PWM que se alejen de este valor ambos motores vayan a la misma velocidad. Además, las ruedas que monta el dispositivo lo hacen muy sensible a los cambios en el pavimento. Ya sea por que dependiendo de por donde circule el deslizamiento cambie, o porqué el suelo tenga pequeñas perturbaciones. Lo que han mostrado los test que se han llevado a cabo para calibrar el robot indican que las perturbaciones en el material producen desviaciones tremendamente grandes en trayectorias rectas. Al fin y al cabo, una desviación angular por muy pequeña que sea, supongamos 5° , se convierte en un error de $2m \cdot \tan(5) = 17,5cm$ que ya no es nada despreciable.

2. Desarrollo Software

2.1. Calibración de servo motores

Tras haber explicado como se ha realizado el montaje de la parte hardware de nuestro robot, comenzamos con la implementación del software que será encargado de controlar el desplazamiento del robot.

En esta primera práctica el principal objetivo es conseguir controlar de manera eficiente el desplazamiento de nuestro robot, así como realizar rotaciones del mismo para distintas orientaciones. Estas acciones son realizadas mediante dos servomotores situados en la parte trasera del robo. Hay que realzar que evidentemente el modelo de desplazamiento que sigue nuestro robot es de tipo diferencial, lo cual supone un control individual de las velocidades de rotación de cada uno de los servomotores. Como además en esta práctica todavía no se ha realizado el montaje de ningún tipo de sensor que nos garantice la correcta rotación de las ruedas según lo planeado, como pudiera ser un encoder, no tendremos realimentación alguna y se realizará consecuentemente un control de la posición del robot en lazo abierto. Provocando que pequeños errores cometidos en los cálculos o montaje se propaguen de manera mucho más fácil influyendo en el correcto desplazamiento del robot.

Tras haber puesto un poco en contexto las limitaciones que vamos a encontrar a la hora de implementar nuestro modelo de desplazamiento, nos centraremos en uno de los principales problemas con los que debemos lidiar a la hora de realizar los diversos cálculos para conseguir un movimiento preciso. Este es la obtención de una velocidad d rotación equivalente para ambas ruedas tractoras. Como ya se ha comentado en la parte del montaje, la velocidad de rotación de los servomotores se controlan mediante señales de tipo PWM ("Pulse Width Modulation"), en las cuales el valor del duty-cycle será el que dictamine el sentido y velocidad de rotación de estos dispositivos.

En teoría la frecuencia de funcionamiento de los servomotores es de 50 Hz, teniendo en cuenta que la frecuencia base de la señal PWM que nos ofrece la Raspberry Pi es de $f_0 = 19,2 \cdot 10^6 \text{ Hz}$, nosotros hemos optado por utilizar unos valores para el Rango de 4000 y para el Clock de 96 de tal manera que se obtuviera, $19,2 \cdot 10^6 / (4000 \cdot 96) = 50 \text{ Hz}$. Usando el datasheet de estos servomotores, encontramos que para los distintos estados de rotación son necesarios los siguientes tiempos en alta de la señal:

Sentido de Rotación	Duty-Cicle (<i>ms</i>)	Value
Horario	1.3	260
Antiorario	1.7	340
Estático	1.5	300

Por lo tanto siguiendo la teoría y habiendo realizado una calibración del potenciómetro de los servomotores, de tal manera que al introducir "value"= 300, los servomotores detuvieran su rotación, se debería cumplir que si asociáramos el valor de 260 al servomotor encargado de la rotación de la rueda derecha y 340 al servomotor encargado de la rueda izquierda, el robot debería realizar un desplazamiento en linea recta sin desviación alguna. Sin embargo, esto no se cumplía en nuestro caso, al introducir estos valores el robot tenía un importante desvío hacia la izquierda fruto de una velocidad mayor de rotación de la rueda derecha. La solución que se nos ocurrió ante este problema consistió en medir la velocidad de rotación de las ruedas para distintos valores, tomando como referencia 20 vueltas, se midieron los tiempos que se tardaba en realizar este número de vueltas para distintos valores:

Value	N	T(s)	W(vueltas/s)
340	20	23,74	0,842
338	20	23,96	0,835
310	20	41,38	0,483
260	20	23,18	0,863
271	20	23,57	0,849
290	20	41,8	0,478

Como vemos las medidas si que reflejan el comportamiento observado, la rueda derecha con el valor teórico 260 se desplaza a mayor velocidad que la rueda izquierda con el valor teórico 340, lo cual provoca la trayectoria desviada hacia la derecha. Nos gustaría comentar que ha pesar de que los resultados parezcan coherentes, las medidas de velocidad de rotación no se deben tomar como una medida exacta e inequívoca,

pues estaríamos pecando de arrogancia, al pensar que somos tan precisos como para parar exactamente un cronómetro cuando la rueda realiza la vuelta 20 y no 100 o 200 milisegundos después, debido a nuestro tiempo de reacción. En resumidas cuentas para las dos velocidades de desplazamiento del robot, se usaron los siguientes valores:

	Rueda derecha	Rueda izquierda
Velocidad alta	271	340
Velocidad media	290	310

2.2. Secciones importantes del software

Posterior a enfrentarnos a este error que nos supuso un autentico quebradero de cabeza, vamos a repasar aquellos fragmentos del código que más interesantes nos parecen, así como ciertos cálculos cuya explicación nos gustaría aclarar, para facilitar la comprensión del software.

Ante la falta de realimentación del estado de giro de las ruedas, pero con el objetivo de simular dicho conteo se usaron hilos para conseguir simular este efecto. Las funciones asociadas a estos hilos siguen la siguiente estructura:

```
1 void *contador_izquierda(void *tiempo) {
2     int t1,t2;
3     float T= *((float*)tiempo); //Obtenemos valor de T
4     while (continuar_hilos) {
5         t1=millis();
6         // Incrementar el contador de la rueda izquierda
7         piLock(0);
8         Ci++;
9         pinlock(0);
10        t2=millis();
11        delay(T* 1000-(t2-t1)); // Utilizar el tiempo T como delay y
                                // corregimos el tiempo invertido en
12    }
13    Ci=0;
14    pthread_exit(NULL);
15 }
```

En las cuales debemos destacar el uso de *piLock* y *piUnlock()*, para bloquear la variable asociada al conteo de franjas y conseguir así que el hilo principal no pueda acceder a ella. Así como el argumento de entrada **tiempo**, que simula el tiempo que tardaría en rotar una franja el servomotor para la velocidad indicada. La variable *continuar_hilos* se usa como semaforo para permitir el uso de los hilos.

Respecto a la función de avance en línea recta del robot las líneas más significativas son:

```
1     printf("En que velocidad desea realizar el desplazamiento.\n
2     Introduzca por consola M(mid) o H(high): ");
3     delay(500);
4     scanf(" %c",&velocidad); //Guardamos la letra
5     if (velocidad == 'M' || velocidad == 'm') {
6         printf("Ha seleccionado velocidad media.\n");
7         i=0;
8     } else if (velocidad == 'H' || velocidad == 'h') {
9         printf("Ha seleccionado velocidad alta.\n");
10        i=1;
11    }
12 } else {
```



```

12         printf("Opcion no valida. Por favor, seleccione L, M o H.\n");
13         return 1; //
14     }
15
16     delay(1000); //pequena pausa para que el usuario lea el mensaje
17     float distancia;
18     printf("Cuanta distancia en cm desea recorrer a dicha velocidad:\n");
19     scanf("%f",&distancia);
20
21     //En este caso como se va a ir solo de frente el conteo ha de ser el
    mismo
22     float R=distancia/perimetro; //Numero de vueltas necesarias para recorrer
    dicha distancia
23     Ct_i=(int)(R*N); //casting a entero del numero de vueltas
24     Ct_d=Ct_i; //En este caso son iguales pues queremos desplazarnos en linea
    recta
25     float W_media[2]={0.54, 0.84}; //Vueltas por segundo
26     float K=20*W_media[i]; //Numero de franjas negras por segundo en funcion
    de la velocidad de rotacion
27     float T=1/K; //Tiempo por contador
28     float V=W_media[i]*M_PI*6.9; //velocidad en m/s

```

Con estas líneas de código conseguimos que al usuario se le pregunte por consola que velocidad desea tener en su desplazamiento al seleccionar una de las dos opciones, la variable *i* toma un valor u otro, lo cual determinará la velocidad que se use para simular el tiempo tardado en recorrer una franja. Y que posteriormente será usado en los hilos como se ha comentado previamente. Además se definen unos contadores, *Ct_i* y *Ct_d*, para limitar las franjas que deben girar las ruedas con el objetivo de recorrer la distancia deseada, esta distancia también será introducida por el usuario.

Posterior a estas líneas de código, se crean los hilos para el conteo de franjas y se usa el comando *pwmWrite(Pin_D,Valor_D[i]);* para comenzar la rotación de los servomotores, como podemos ver con la variable *i*, se elegirá uno de los dos valores a escribir en el PWM. Y se comienza a ejecutar un bucle infinito:

```

1     while(1){
2         // Leer el contador de la rueda izquierda
3         piLock(0);
4         int contador_izq = Ci;
5         piUnlock(0);
6         // Leer el contador de la rueda derecha
7         piLock(1);
8         int contador_der = Cd;
9         piUnlock(1);
10        // Mostrar los valores de los contadores
11        printf("Contador izquierda: %d, Contador derecha: %d\n",
            contador_izq, contador_der);
12
13        // Logica para detener los motores si se supera el limite Ct en ambos
    contadores
14
15        if (contador_izq >= Ct_i && contador_der >= Ct_d) {
16
17            // Detener los motores
18
19            pwmWrite(Pin_I, 0);

```

```

20
21     pwmWrite(Pin_D, 0);
22
23     printf("Se paro el giro de ambas ruedas.\n");
24     continuar_hilos=0;
25     pthread_join(hilo_derecha,NULL); //Nos aseguramos de que se
        cierre el hilo
26     pthread_join(hilo_izquierda,NULL);
27
28     break; // Salir del bucle
29
30 }
31 // Logica para detener cada motor individualmente si se supera el
    limite Ct
32
33     if (contador_izq >= Ct_i) {
34         pwmWrite(Pin_I, 0);
35         printf("Se paro el giro de la rueda izquierda.\n");
36     }
37
38     if (contador_der >= Ct_d) {
39         pwmWrite(Pin_D, 0);
40         printf("Se paro el giro de la rueda derecha.\n");
41     }

```

Con el cual vamos actualizando el valor del contador de franjas hasta que se cumple la condición en la cual se han alcanzado los conteos de franjas artificiales deseados, uso el termino artificial, pues realmente no se están contando franjas por ningún tipo de sensor. En realidad estamos simulando este tiempo con una serie de retardos. Una vez alcanzados dichos contadores totales se para la rueda y se cierran los hilos. Como podemos ver el uso de *piLock()* y *piUnlock()* vuelve a ser necesario para en este caso dejar la variable bloqueada en el hilo principal mientras se realiza la comprobación que no se hayan superado los valores limite para estos contadores.

Respecto a la función de rotación del robot, nos gustaría destacar los cálculos que realiza el programa para poder estimar el tiempo necesario que se deben de tener funcionando los servomotores, para conseguir una rotación del ángulo que haya introducido el usuario. Las cuentas son similares a las del desplazamiento en línea recta, pero con la peculiaridad de que en este caso se usa la distancia del arco que ha de recorrer el robot, siendo este el producto entre el eje del robot y el angulo que se desea rotar en radianes.

```

1     float W_rotacion=0.54;
2     float K=20*W_rotacion; //Numero de franjas negras por segundo en funcion
        de la velocidad de rotacion
3
4     float T=1/(K*2); //Tiempo por contador dividimos entre dos pues la
        velocidad angular se duplica al girar ambas ruedas
5
6     int Value_rot_D[2]={289,311};
7     int Value_rot_I[2]={289,311};
8     int j;
9     delay(100);
10
11     scanf(" %c", &rotacion);
12
13     if( rotacion == 'i' || rotacion == 'I'){

```

```

14
15     printf("Con que angulo se desea rotar a izquierdas:\n");
16
17     float angulo_izquierda;
18
19     scanf("%f",&angulo_izquierda);
20
21     S=eje_robot*angulo_izquierda*M_PI/180;
22     float R=S/perimetro;//Numero de vueltas necesarias para recorrer
        dicha distancia
23     Ct_d=(int)(R*N);//casting a entero del numero de franjas que ha de
        recorrer la rueda derecha
24     Ct_i=Ct_d;// Como queremos girar a izquierdas
25     j=0;//Se seleccionan los valores de rotacion asociados al giro a
        izquierdas
26 }
27
28 else if ( rotacion == 'd' || rotacion == 'D'){
29
30     printf("Con que angulo se desea rotar a derechas:\n");
31
32     float angulo_derecha;
33
34     scanf("%f",&angulo_derecha);
35     S=eje_robot*angulo_derecha*M_PI/180;
36     float R=S/perimetro;//Numero de vueltas necesarias para recorrer
        dicha distancia
37     Ct_i=(int)(R*N);//casting a entero del numero de franjas que ha de
        recorrer la rueda izquierda para completar la rotacion
38     Ct_d=Ct_i;// Como queremos girar a izquierdas
39     j=1;
40
41
42 }
43
44 else {
45
46     printf("Opcion no valida. Por favor, pulse I o D.\n");
47
48     return 1;//
49
50 }

```

Como vemos en este caso al seleccionar el sentido de rotación a izquierdas o a derechas, se fija un valor para la variable `j`, que elegirá el par de valores introducidos en las funciones que nos proporciona `wiringPi` para generar PWM en los servomotres izquierdo y derecho. También se ha de destacar el factor 2 en el cálculo del tiempo por contador, que como se explica en el propio código viene del uso de dos servomotores, lo cual duplica la velocidad angular. Posterior a la selección del sentido de rotación y cálculo del arco, se crean los hilos necesarios para el conteo de franjas y se inicia un bucle infinito de igual manera que para el caso del avance del robot, con el objetivo de que las ruedas alcancen los contadores límites y una vez cumplida dicha condición se paren los motores.